

Algorithmes - Correction

Muhammad ibn Musa al-Khwarizmi (vers 780 – 850) était un mathématicien, astronome et géographe persan, dont les travaux ont profondément influencé le développement des mathématiques en Occident. Son ouvrage le plus célèbre, *Kitab al-Jabr wa-l-Muqabala*, introduit les méthodes systématiques pour résoudre des équations linéaires et quadratiques, donnant naissance au mot « algèbre » (al-jabr). Mais al-Khwarizmi est également célèbre pour ses méthodes de calcul : il décrivait des procédures étape par étape pour résoudre des problèmes arithmétiques et algébriques. Ces procédures sont aujourd'hui reconnues comme les premiers algorithmes, terme dérivé de la latinisation de son nom (Algoritmi).

Nicolas VACELET – Université Marie et Louis Pasteur

14 Octobre 2025

Ordres de grandeur

1. **Qu'est-ce qui est le plus grand, 10^{100} ou 100^{10} ?**

Pour comparer ces deux nombres, nous pouvons les exprimer avec la même base.

$$100^{10} = (10^2)^{10} = 10^{2 \times 10} = 10^{20}$$

Puisque $100 > 20$, on a :

$$10^{100} > 100^{10}$$

10^{100} est un Googol, un nombre bien plus grand que 10^{20} .

2. **Combien de chiffres comporte 2^n en écriture décimale ?**

Le nombre de chiffres d'un entier X est donné par $\lfloor \log_{10}(X) \rfloor + 1$. Pour $X = 2^n$, le nombre de chiffres est :

$$\lfloor \log_{10}(2^n) \rfloor + 1 = \lfloor n \cdot \log_{10}(2) \rfloor + 1$$

Comme $\log_{10}(2) \approx 0.30103$, le nombre de chiffres de 2^n est approximativement $0.301n + 1$.

3. **Estimer l'ordre de grandeur de $n!$ pour $n = 20$ et comparer à 2^n et n^n .**

Nous utilisons l'approximation de Stirling pour $n!$: $n! \approx \sqrt{2\pi n} \left(\frac{n}{e}\right)^n$. Pour $n = 20$ (avec $e \approx 2.718$ et $\pi \approx 3.141$) :

$$20! \approx \sqrt{2\pi \cdot 20} \left(\frac{20}{e}\right)^{20} \approx \sqrt{125.66} (7.358)^{20} \approx 11.2 \times (1.64 \times 10^{17}) \approx 1.84 \times 10^{18}$$

La valeur exacte est $20! \approx 2.43 \times 10^{18}$. L'approximation est donc correcte.

Comparons avec 2^n et n^n pour $n = 20$:

- $2^{20} = (2^{10})^2 = (1024)^2 \approx (10^3)^2 = 10^6 = \mathbf{1.048 \times 10^6}$
- $20^{20} = (2 \times 10)^{20} = 2^{20} \times 10^{20} \approx 1.05 \times 10^6 \times 10^{20} = \mathbf{1.05 \times 10^{26}}$

On observe la hiérarchie : $2^{20} \ll 20! \ll 20^{20}$.

4. **Vérifier si n^5 est négligeable devant 2^n pour $n = 50$.**

On veut comparer 50^5 et 2^{50} .

- $50^5 = (5 \times 10)^5 = 5^5 \times 10^5 = 3125 \times 10^5 = \mathbf{3.125 \times 10^8}$
- $2^{50} = 2^{10 \times 5} = (2^{10})^5 = (1024)^5 \approx (10^3)^5 = 10^{15}$. Plus précisément, $2^{50} \approx \mathbf{1.126 \times 10^{15}}$.

Clairement, 3.125×10^8 est très inférieur à 1.126×10^{15} . On peut donc considérer que n^5 est négligeable devant 2^n pour $n = 50$.

Comparaison de complexités

1. $3^n = O(2^n)$?

Non. Par définition, $f(n) = O(g(n))$ s'il existe une constante $C > 0$ et un entier n_0 tels que pour tout $n \geq n_0$, on a $f(n) \leq C \cdot g(n)$. Ici, nous aurions besoin que $3^n \leq C \cdot 2^n$ pour n assez grand. Cela se réécrit $\left(\frac{3}{2}\right)^n \leq C$. Or, la fonction $\left(\frac{3}{2}\right)^n$ est une exponentielle de base > 1 , elle tend donc vers l'infini. Elle ne peut pas être bornée par une constante C .

2. $\log 3^n = O(\log 2^n)$?

Oui. Simplifions les expressions :

- $\log 3^n = n \log 3$
- $\log 2^n = n \log 2$

Nous devons vérifier s'il existe C et n_0 tels que $n \log 3 \leq C \cdot (n \log 2)$ pour $n \geq n_0$. En divisant par n (pour $n > 0$), on obtient $\log 3 \leq C \cdot \log 2$. On peut choisir $C = \frac{\log 3}{\log 2} \approx 1.585$. Cette constante existe et est positive. La relation est donc vraie pour tout $n \geq 1$.

3. **Montrer que $n^2 = O(n^3)$ mais $n^3 \neq O(n^2)$.**

- $n^2 = O(n^3)$: Nous devons trouver $C > 0$ et n_0 tels que $n^2 \leq C \cdot n^3$ pour $n \geq n_0$. En divisant par n^2 (pour $n > 0$), on obtient $1 \leq C \cdot n$. Si nous choisissons $C = 1$, l'inégalité $1 \leq n$ est vraie pour tout $n \geq 1$. Donc, $n^2 = O(n^3)$.
- $n^3 \neq O(n^2)$: Supposons par l'absurde que $n^3 = O(n^2)$. Alors il existerait $C > 0$ et n_0 tels que $n^3 \leq C \cdot n^2$ pour $n \geq n_0$. En divisant par n^2 , on obtient $n \leq C$. Cette inégalité est fausse pour tout entier $n > C$, ce qui contredit le fait qu'elle doit être vraie pour tous les $n \geq n_0$. La supposition est donc fausse.

4. Comparer 2^n et $n!$ pour n grand.

La factorielle $n!$ croît beaucoup plus rapidement que l'exponentielle 2^n . Pour le montrer, on peut regarder le ratio des termes consécutifs :

$$\frac{(n+1)!/2^{n+1}}{n!/2^n} = \frac{(n+1)!}{n!} \cdot \frac{2^n}{2^{n+1}} = (n+1) \cdot \frac{1}{2} = \frac{n+1}{2}$$

Ce ratio est supérieur à 1 dès que $n+1 > 2$, c'est-à-dire $n > 1$. Comme le ratio tend vers l'infini, $n!$ domine largement 2^n . On peut donc écrire $2^n = o(n!)$, ce qui implique $2^n = O(n!)$.

Impact de la vitesse du processeur

Soit M la taille maximale du problème résoluble en 1 minute. Le nombre d'opérations est $f(M)$. Un processeur 100 fois plus rapide peut effectuer $100 \cdot f(M)$ opérations dans le même temps. Soit M' la nouvelle taille maximale. On a donc $f(M') = 100 \cdot f(M)$.

- **Algorithme linéaire** $O(n)$: $f(n) = c \cdot n$.

Donc, $f(M') = c \cdot M' = 100 \cdot c \cdot M \implies M' = 100M$. La taille du problème est multipliée par 100.

- **Algorithme quadratique** $O(n^2)$: $f(n) = c \cdot n^2$.

$f(M') = c \cdot (M')^2 = 100 \cdot c \cdot M^2 \implies (M')^2 = 100M^2 \implies M' = 10M$. La taille est multipliée par 10.

- **Algorithme polynomial** $O(n^k)$: $f(n) = c \cdot n^k$.

$c \cdot (M')^k = 100 \cdot c \cdot M^k \implies (M')^k = 100M^k \implies M' = \sqrt[k]{100} \cdot M$.

- **Algorithme exponentiel** $O(2^n)$: $f(n) = c \cdot 2^n$.

$c \cdot 2^{M'} = 100 \cdot c \cdot 2^M \implies 2^{M'} = 100 \cdot 2^M$. Prenons le log en base 2 : $\log_2(2^{M'}) = \log_2(100 \cdot 2^M) \implies M' = \log_2(100) + \log_2(2^M)$. $M' = M + \log_2(100) \approx M + 6.64$. On ne gagne qu'une augmentation additive.

- **Question supplémentaire : que se passe-t-il pour $O(n!)$?** $f(n) = c \cdot n!$.

$(M')! = 100 \cdot M!$. Il est difficile d'exprimer M' simplement en fonction de M . Cependant, on voit que l'augmentation est très faible. Par exemple, si $M = 5$, $M! = 120$. On cherche M' tel que $(M')! \approx 100 \times 120 = 12000$. Comme $7! = 5040$ et $8! = 40320$, la nouvelle taille M' sera entre 7 et 8. Le gain est encore plus faible qu'en exponentiel.

Choix optimal entre algorithmes

Complexités : $A_1(n) = 2n^2$, $A_2(n) = 10n$, et $A_3(n) = 40 \log_2 n$.

1. Pour $n = 10$, quel est l'algorithme le plus rapide ?

- $A_1(10) = 2 \cdot 10^2 = 200$
- $A_2(10) = 10 \cdot 10 = 100$
- $A_3(10) = 40 \log_2(10) \approx 40 \times 3.32 = 132.8$

Pour $n = 10$, l'algorithme le plus rapide est $A_2(n) = 10n$.

2. Pour $n = 1000$, quel est l'algorithme le plus efficace ?

- $A_1(1000) = 2 \cdot 1000^2 = 2,000,000$
- $A_2(1000) = 10 \cdot 1000 = 10,000$
- $A_3(1000) = 40 \log_2(1000) \approx 40 \times 9.97 = 398.8$

Pour $n = 1000$, l'algorithme le plus efficace est $A_3(n) = 40 \log_2 n$.

3. Quelle stratégie utiliseriez-vous pour choisir l'algorithme en fonction de la taille n ?

La meilleure stratégie est de déterminer les points de croisement où une complexité devient plus avantageuse qu'une autre.

- (a) **Comparer $A_2(10n)$ et $A_3(40 \log_2 n)$** : L'algorithme logarithmique deviendra plus rapide que le linéaire pour des valeurs de n relativement petites. On résout $10n = 40 \log_2 n \implies n = 4 \log_2 n$. Par essais, on voit que pour $n = 1$, $1 > 0$, mais pour $n = 2$, $2 < 4$. Le croisement se situe entre $n = 1$ et $n = 2$. Pour $n \geq 2$, l'algorithme logarithmique est meilleur.
- (b) **Comparer $A_1(2n^2)$ et $A_2(10n)$** : On résout $2n^2 = 10n \implies 2n = 10 \implies n = 5$. Pour $n > 5$, l'algorithme linéaire A_2 est plus rapide que le quadratique A_1 .

Stratégie :

Choisir l'algorithme selon la taille des données :

- Pour des petites tailles, un algorithme plus simple ou quadratique peut suffire (coût d'implémentation faible).
- Pour des grandes tailles, il faut privilégier les complexités plus faibles comme $O(n)$ ou $O(\log n)$.

Variabilité de la complexité

Peut-on avoir un algorithme dont le pire cas est exponentiel, mais certaines entrées sont rapides ?

Oui, c'est tout à fait possible et même courant. La complexité dans le pire des cas ($O(2^n)$) garantit une borne supérieure sur le temps d'exécution, mais ne dit rien sur le meilleur cas ou le cas moyen.

- **Exemple concret** : L'algorithme du **Simplex** pour la programmation linéaire. En théorie, sa complexité dans le pire cas est exponentielle sur des exemples pathologiques (comme le cube de Klee-Minty). Cependant, en pratique, sur la grande majorité des problèmes du monde réel, il se comporte de manière très efficace, souvent en temps polynomial.

Un autre exemple plus simple est un algorithme de recherche de solution par force brute. Prenons la recherche d'un sous-ensemble d'entiers dont la somme est nulle (Subset Sum Problem).

- **Pire cas** : L'algorithme doit tester les 2^n sous-ensembles possibles. La complexité est $O(2^n)$.
- **Meilleur cas** : Si l'entrée est '[5, -5, 10, 20]', l'algorithme peut trouver le sous-ensemble '[5, -5]' très rapidement et s'arrêter. Le temps d'exécution est alors très court.
- **le parcours d'un arbre de recherche (backtracking)**. Dans le pire cas, toutes les branches sont explorées : complexité $O(2^n)$. Mais si la solution est trouvée tôt ou si des heuristiques coupent les branches, le temps peut être linéaire ou quadratique.

- Autre exemple : **le tri rapide (quicksort)**. Il est en moyenne $O(n \log n)$, mais peut devenir $O(n^2)$ dans le pire cas.

Combinaison de sommes

La fonction compte le nombre de triplets d'entiers non-négatifs (i, j, k) tels que $i + 2j + 5k = n$.

- **Complexité** : La version initiale est bien en $O(n^3)$. Le nombre d'itérations est proportionnel à $\sum_{i=0}^n \sum_{j=0}^{(n-i)/2} \sum_{k=0}^{(n-i-2j)/5} 1$, ce qui correspond au volume d'un tétraèdre dans l'espace (i, j, k) , dont le volume est proportionnel à n^3 .
- **Optimisation** : On peut éliminer la boucle la plus interne. Pour des valeurs données de i et j , la valeur de k est fixée par l'équation : $k = (n - i - 2j)/5$. Il suffit de vérifier si cette valeur de k est un entier non-négatif. La boucle sur k et le test 'if' sont remplacés par un seul test de divisibilité.

```
fonction f_optimisee(n: entier): entier
debut
somme <- 0
pour i variant de 0 a n
pour j variant de 0 a (n-i)/2
si (n - i - 2j) >= 0 et (n - i - 2j) mod 5 = 0 alors
somme <- somme + 1
fin si
fin pour
fin pour
retourner somme
fin
```

Cette fonction a deux boucles imbriquées, sa complexité est donc $O(n^2)$.

- **Exercice supplémentaire : solutions avec $i \leq j \leq k$** Il faut ajouter les contraintes $i \leq j$ et $j \leq k$ à la recherche. En substituant $k = (n - i - 2j)/5$, la deuxième contrainte devient $j \leq (n - i - 2j)/5$, ce qui se simplifie en $5j \leq n - i - 2j \implies 7j \leq n - i \implies j \leq (n - i)/7$. La fonction modifiée devient :

```
fonction f_contrainte(n: entier): entier
debut
somme <- 0
// La contrainte i <= j <= k implique i+2i+5i <= n => 8i <= n
pour i variant de 0 a n/8
// La contrainte j <= k implique j <= (n-i)/7
pour j variant de i a (n-i)/7
si (n - i - 2j) mod 5 = 0 alors
k <- (n - i - 2j) / 5
// Il faut encore vérifier que j <= k
si j <= k alors
somme <- somme + 1
fin si
fin si
```

```
fin pour
fin pour
retourner somme
fin
```

La complexité reste en $O(n^2)$ car il y a toujours deux boucles imbriquées, bien que les bornes soient plus serrées, ce qui rend l'algorithme plus rapide en pratique.

Pour ne compter que les solutions avec $i \leq j \leq k$, il faut ajouter une condition dans la boucle :

si $i \leq j$ et $j \leq k$.

Cela réduit le nombre de combinaisons explorées, mais la complexité reste du même ordre.